Database Report on the Dollar Flu outbreak in Manhattan.

3428295

***Abstract*—This report will include information on the scenario, the process of creating the tables, importing the data, queries to find insights on the data and complexity.**

I. Introduction, Scenario and Data Collection

In 2001, a high-level bio-terror attack simulation named operation dark winter was conducted in Maryland, United States to model a fictional scenario detailing a smallpox attack on the country starting from Oklahoma City [1]. It aimed to raise awareness of the possibility of a biological attack, understand how overwhelmed emergency response would be and expose flaws in co-ordination among government officials.

The scenario is heavily inspired by the operation and will convey a snapshot covering a month after an outbreak of a genetically-engineered supervirus called The Dollar Flu in Manhattan, New York without quarantines or medical efforts. The virus is from the fictional work The Division by Tom Clancy which contains traits from several other viruses such as Ebola and Hantavirus. It has an infection rate of 95 percent and a recovery of 5 percent which will heavily influence the data for the snapshot[2]. The blocks of each neighborhood will be approximated.

To simulate the snapshot, data will be collected from open data sources detailing statistics on the populations of its neighborhoods. Sources will include data from the official government website [3] and neighborhood statistics from WorldAtlas [4]. Data for the number of infected and recovered cases among the population of each block will be generated based on the infection and recovery rate of the Dollar Flu (95 percent become infected, the rest recover).

II. Database Design and Decisions

The ER diagram visualizes our database and shows the decisions made. There will be 4 entities each represented by a single table. Each entity has its own primary key to make sure no duplicates exist and each entity (except transmission) will be in third normal form so every non key attribute is tied to the primary key. Neighborhood will have its own table rather than being part of the blocks table as this will remove the transitive dependency chain of block ID determining neighborhood ID determining neighborhood name. Transmission is its own table and works as a compound key as one event can have several different methods with one method per row leading to different combinations of event and transmission without repetition of one. Storing the transmission methods as a list for each event would break 1NF.

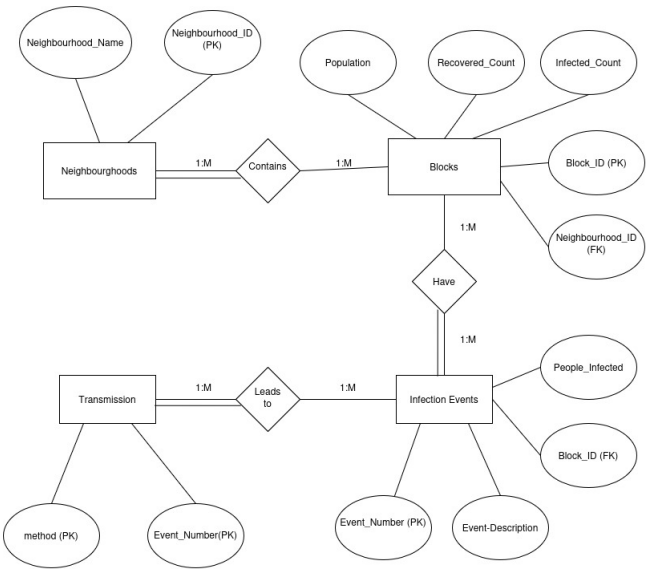


Fig. 1. An ER diagram of the database

The tables for cityblocks and infection events are all 3NF to make it obvious one table describes the blocks of the city and one describes the outbreaks, so that no mixed or unrelated attributes are included. All attributes for each table will include the NOT NULL option in sql so no record is incomplete.

III. Table Creation

We will first create the neighbourhood table. Neighbourhood ID will be used as the primary key to make sure each neighbourhood. The name of the neighbourhood will also be included:

```
    CREATE TABLE neighbourhood (
Neighbourhood_ID  INT  NOT NULL,
Neighbourhood_Name  varchar(50)  NOT NULL,
PRIMARY KEY(Neighbourhood_Id));
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	Neighbourhood_ID	int			No	None			Change Drop More
2	Neighbourhood_Name	varchar(50)	utf8mb4_0900_ai_ci		No	None			Change Drop More

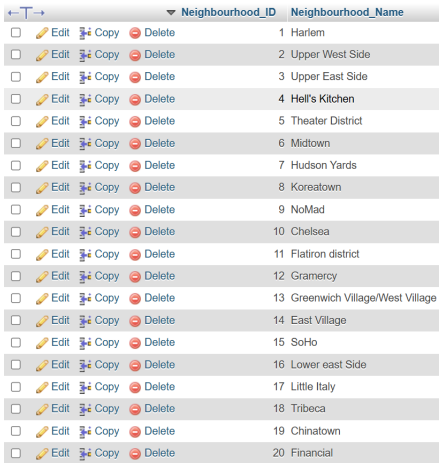
Fig. 2. The neighbour table's structure

To import data we will use the following code:

```
-- data can only be loaded from
wamp folder
```

```
LOAD DATA INFILE 'C:/wamp64/tmp/
Neighbourhoods.csv'
INTO TABLE neighbourhood
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS;
```

This code will be used to import every single CSV onto the tables. It will look identical with the only difference being the name of the CSV. The names of the CSVs will be from the ER diagrams for each respective entity.

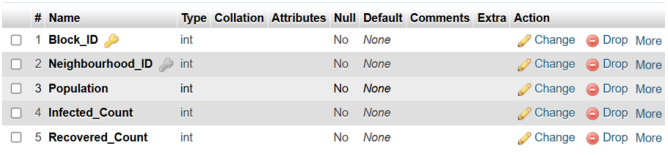


	Neighbourhood_ID	Neighbourhood_Name
Edit Copy Delete	1	Harlem
Edit Copy Delete	2	Upper West Side
Edit Copy Delete	3	Upper East Side
Edit Copy Delete	4	Hell's Kitchen
Edit Copy Delete	5	Theater District
Edit Copy Delete	6	Midtown
Edit Copy Delete	7	Hudson Yards
Edit Copy Delete	8	Koreatown
Edit Copy Delete	9	NoMad
Edit Copy Delete	10	Chelsea
Edit Copy Delete	11	Flatiron district
Edit Copy Delete	12	Gramercy
Edit Copy Delete	13	Greenwich Village/West Village
Edit Copy Delete	14	East Village
Edit Copy Delete	15	SoHo
Edit Copy Delete	16	Lower east Side
Edit Copy Delete	17	Little Italy
Edit Copy Delete	18	Tribeca
Edit Copy Delete	19	Chinatown
Edit Copy Delete	20	Financial

Fig. 3. Neighbourhood data imported

The second table we will make is the city block table. NeighbourhoodID will be its foreign key and we will have blockID as its primary key and will also contain numbers on population as well as the number of people currently infected or recovered after 1 month.

```
    CREATE TABLE CityBlock (
Block_ID  INT  NOT NULL,
 Neighbourhood_ID  INT  NOT NULL,
Population  INT  NOT NULL,
 Infected_Count  INT  NOT NULL,
Recovered_Count  INT  NOT NULL,
 PRIMARY KEY (Block_ID),
 FOREIGN KEY(Neighbourhood_ID)
 REFERENCES neighbourhood
 (Neighbourhood_ID) );
```

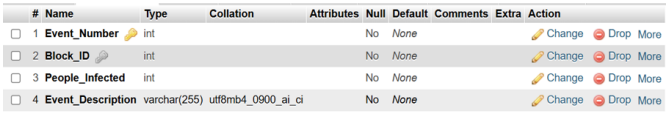


#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	Block_ID	int			No	None			Change Drop More
2	Neighbourhood_ID	int			No	None			Change Drop More
3	Population	int			No	None			Change Drop More
4	Infected_Count	int			No	None			Change Drop More
5	Recovered_Count	int			No	None			Change Drop More

Fig. 4. Cityblock's table structure

We will create the infection event table. BlockID will act as the foreign key and each event will have its own number as its code along with an attribute describing the event and another for the number of people who were initially infected during after the event. We will do this with the following code:

```
CREATE TABLE InfectionEvent (
Event_Number  INT  NOT NULL,
 Block_ID  INT  NOT NULL,
People_Infected  INT  NOT NULL,
Event_Description  VARCHAR(255)  NOT NULL,
PRIMARY KEY (Event_Number),
FOREIGN KEY(Block_ID) REFERENCES
CityBlock (Block_ID) );
```

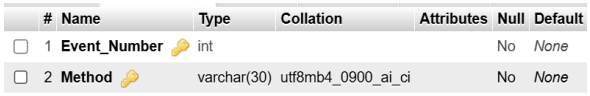


#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	Event_Number	int			No	None			Change Drop More
2	Block_ID	int			No	None			Change Drop More
3	People_Infected	int			No	None			Change Drop More
4	Event_Description	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More

Fig. 5. Event's table structure

The following code will be used to create the transmission table. It will have the event number along with a method of transmission for the virus.

```
CREATE TABLE transmission (
    Event_Number    INT  NOT NULL,
    Method VARCHAR(30)   NOT NULL,
    -- following will act as our compound key
    PRIMARY KEY (Event_Number, Method),
    FOREIGN KEY (Event_Number)
    REFERENCES InfectionEvent (Event_Number));
```



#	Name	Type	Collation	Attributes	Null	Default
1	Event_Number	int			No	None
2	Method	varchar(30)	utf8mb4_0900_ai_ci		No	None

Fig. 6. The result of the code

IV. Insert Statements

This section will include examples of how we might want to insert new data into each of our tables. The statements will insert instances only to show that the statements work. they will be removed after.

We will use the following code to add a new neighbourhood:

```
   INSERT INTO neighbourhood(Neighbourhood_ID,
   Neighbourhood_Name)
VALUES (21, "random_neighbourhood");
-- The name is just an example since every
neighbourhood from manhattan is written down.
```



Edit Copy Delete	19	Chinatown
Edit Copy Delete	20	Financial
Edit Copy Delete	21	random_neighbourhood

Fig. 7. The new neighbourhood as an instance

For cityblock, we will insert a new block into the financial neighbourhood.

```
   INSERT INTO cityblock(Block_ID,
```

```
    Neighbourhood_ID,
    Population,  Infected_Count,
    Recovered_Count)
VALUES (55, 20, 20000, 19000, 1000);
```

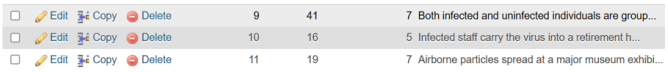


Edit Copy Delete	52	19	45220	42959	2261
Edit Copy Delete	53	20	16345	15528	817
Edit Copy Delete	54	20	19654	18671	983
Edit Copy Delete	55	20	20000	19000	1000

Fig. 8. The new block inserted in the table

We will add a new infection outbreak for the event table with a description which is a realistic possibility in the scenario:

```
    INSERT INTO infectionevent (
    Event_Number,
    Block_ID, People_Infected,
    Event_Description)
VALUES (11, 19, 7, "Airborne particles
spread at a major museum exhibit");
```

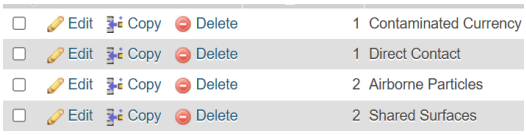


Edit Copy Delete	9	41	7	Both infected and uninfected individuals are group...
Edit Copy Delete	10	16	5	Infected staff carry the virus into a retirement h...
Edit Copy Delete	11	19	7	Airborne particles spread at a major museum exhibi...

Fig. 9. The new event instance

If we wanted to add a new transmission method for an existing outbreak event we would use the following example:

```
    INSERT INTO transmission(Event_Number,
    Method)
VALUES(2, "Shared Surfaces");
```



Edit Copy Delete	1	Contaminated Currency
Edit Copy Delete	1	Direct Contact
Edit Copy Delete	2	Airborne Particles
Edit Copy Delete	2	Shared Surfaces

Fig. 10. The new instance of transmission event in the table

V. SQL Queries

The following section will include sql queries used to provide useful insights from the data.

The first query will make use of the LIKE operator to instances with certain words in the neighbourhood table:

```
SELECT * FROM neighbourhood WHERE
Neighbourhood_Name LIKE "%Side";
-- Neighbourhood names ending in side
```



	Neighbourhood_ID	Neighbourhood_Name
Edit Copy Delete	2	Upper West Side
Edit Copy Delete	3	Upper East Side
Edit Copy Delete	16	Lower east Side

Fig. 11. The result of query 1

We used the this query to easily identify the neighbourhood names with the word ”side”. This makes it simpler for searching the data we want out of the 20 instances.

The next query will make use of the JOIN operator to select data from multiple tables. We will use the tables infectionevent and cityblock.

```
SELECT infectionevent.Block_ID, Event_Number,
People_Infected, Event_Description,
Infected_Count
FROM infectionevent
 INNER JOIN cityblock
 ON infectionevent.Block_ID = cityblock.Block_ID
```

Block_ID	Event_Number	People_Infected	Event_Description	Infected_Count
13	1	10	Infected dollar bills used in a local mall, beginn...	40713
14	2	5	Infected individuals from the original infection e...	42624
24	3	8	several infected commute on the subway leading to ...	20805
51	4	12	Street market vendors come into contact with infec...	52146
3	5	6	The infection spreads through airborne particles a...	27510
53	6	15	Infected cash passes through a busy ATM area	15528
22	7	20	Airborne particles spread the infection at a theat...	17210
33	8	4	Staff and other patients are exposed to the virus ...	19824
41	9	7	Both infected and uninfected individuals are group...	22621
16	10	5	Infected staff carry the virus into a retirement h...	38560

Fig. 12. The result of query 2

The purpose of this query was to compare the number of people who were infected at the cityblocks where the initial outbreaks started with the current number of infected cases for those same blocks 1 month later

The third query will use the not equal operator to evaluate if values on its left are bigger than the values on the right. We will use the cityblock table.

```
SELECT Block_ID, Neighbourhood_ID, Population,
Infected_Count
FROM cityblock
WHERE Infected_Count > 40000;
-- WHERE is used as a filter to show the blocks
with the highest infected count
```

	Block_ID	Neighbourhood_ID	Population	Infected_Count
Edit Copy Delete	9	2	48769	46331
Edit Copy Delete	10	2	43785	41596
Edit Copy Delete	11	2	46087	43783
Edit Copy Delete	12	2	44789	42550
Edit Copy Delete	13	2	42856	40713
Edit Copy Delete	14	3	44867	42624
Edit Copy Delete	15	3	42890	40746
Edit Copy Delete	17	3	48356	45938
Edit Copy Delete	18	3	46899	44554
Edit Copy Delete	51	19	54890	52146
Edit Copy Delete	52	19	45220	42959

Fig. 13. The result of query 3

We now see a subset of blocks which have high infected counts of over 40000.

The next query will use GROUP BY on the cityblock table to group blocks into their respective neighbourhoods and will also make use of aggregate functions.

```
SELECT Neighbourhood_ID,
SUM(Infected_Count),
COUNT(Block_ID)
FROM cityblock
GROUP BY Neighbourhood_ID;
```

Neighbourhood_ID	SUM(Infected_Count)	COUNT(Block_ID)
1	247026	8
2	214973	5
3	212422	5
4	50795	3
5	17210	1
6	98102	5
7	27876	2
8	2565	1
9	19714	2
10	59696	3
11	17656	1
12	49349	2
13	21229	2
14	65791	3
15	12887	2
16	47811	2
17	8545	1
18	20432	2
19	95105	2
20	34199	2

Fig. 14. The result of query 4

We use the following code to count the total number of people infected and number of blocks as an insight on how the infection count and size differs from each neighbourhood.

The fifth query will return a subset of data which matches or is part of a list of values with the IN operation. We will use it on the cityblock table.

```
SELECT Block_ID, Neighbourhood_ID,
Population, Infected_Count
FROM cityblock
WHERE Neighbourhood_ID IN (1, 2, 3);
```

	Block_ID	Neighbourhood_ID	Population	Infected_Count
Edit Copy Delete	1	1	32250	30638
Edit Copy Delete	2	1	38543	36616
Edit Copy Delete	3	1	28958	27510
Edit Copy Delete	4	1	35986	34187
Edit Copy Delete	5	1	31435	29863
Edit Copy Delete	6	1	30586	29057
Edit Copy Delete	7	1	27688	26304
Edit Copy Delete	8	1	34580	32851
Edit Copy Delete	9	2	48769	46331
Edit Copy Delete	10	2	43785	41596
Edit Copy Delete	11	2	46087	43783
Edit Copy Delete	12	2	44789	42550
Edit Copy Delete	13	2	42856	40713
Edit Copy Delete	14	3	44867	42624
Edit Copy Delete	15	3	42890	40746
Edit Copy Delete	16	3	40589	38560
Edit Copy Delete	17	3	48356	45938
Edit Copy Delete	18	3	46899	44554

Fig. 15. The result of query 5

The following shows all the blocks which part of neighbourhoods 1, 2 and 3.

The sixth and final query will be using the UPDATE function to modify existing values. Here are the values which will be modified:

	Block_ID	Neighbourhood_ID	Population	Infected_Count	Recovered_Count
Edit Copy Delete	1	1	32250	30638	1613
Edit Copy Delete	15	3	42890	40746	2145
Edit Copy Delete	51	19	54890	52146	274

Fig. 16. values which need to be updated

The image shows us that the sum of the two counts is one value higher than the blocks population. We will need to de increment the recovery count so that the values are accurate to the blocks population and because infection count is the key attribute of our scenario with recovery count being the remainder of people who weren't affected by the virus.

```
UPDATE cityblock
SET Recovered_Count = Recovered_Count - 1
-- de-incrementing the recovered count for
certain rows
WHERE Block_ID IN (1, 15, 51);
```

	Block_ID	Neighbourhood_ID	Population	Infected_Count	Recovered_Count
Edit Copy Delete	1	1	32250	30638	1613
Edit Copy Delete	15	3	42890	40746	2145
Edit Copy Delete	51	19	54890	52146	274

Fig. 17. The result of query 6

VI. Discussion of Complexity

The database holds one infected count per block and is supposed to be a snapshot 1 month after the outbreaks but doesn't give a clear image as to how the initial infected cases,ranging from 7 to 20, changed and grew to significantly larger numbers. To keep track of this we would need an attribute to represent time such as day which would be a significant task as there would be 54 blocks each containing 30 days worth of data. What also makes this task more complex is that SQL is less useful for this task and is more suitable for querying static data.

A more suitable way to keep track of changes over time is to run this model in Python which includes loops and iteration. SQL is useful

References

[1] "Dark Winter Johns Hopkins Center for Health Security," *centerforhealthsecurity.org*. https://centerforhealthsecurity.org/our-work/tabletop-exercises/dark-winter-a-training-tabletop-exercise

[2] Green poison — the division wiki — fandom, https://thedivision.fandom.com/wiki/Green_Poison

[3] "NYC Population by Neighborhood — NYC Open Data," *Cityofnewyork.us*, 2018. https://data.cityofnewyork.us/City-Government/NYC-Population-by-Neighborhood/t79z-mgxr

[4] "Manhattan Neighborhoods by Population," *WorldAtlas*, Feb. 20, 2019. https://www.worldatlas.com/articles/manhattan-neighborhoods-by-population.html (accessed Oct. 25, 2026).